What is an expression?

Lean represents terms, including types and proofs, as trees of type `Expr`.

```
inductive Expr where
  | bvar    :  Nat → Expr                                  variable under a binder
  | fvar    :  FVarId → Expr                               local variable
  | mvar    :  MVarId → Expr                               metavariable
  | sort    :  Level → Expr                                sorts: Prop, Type, ...
  | const   :  Name → List Level → Expr                    declared constant
  | app     :  Expr → Expr → Expr                          function application
  | lam     :  Name → Expr → Expr → BinderInfo → Expr      function abstraction
  | forallE : Name → Expr → Expr → BinderInfo → Expr       dependent function type
  | letE    :  Name → Expr → Expr → Expr → Bool → Expr     local definition
  | lit     :  Literal → Expr                              number or string literal
  | mdata   :  MData → Expr → Expr                         metadata around a term
  ...
```

Metavariables: unfinished expressions

`Expr.mvar : MVarId → Expr` represents a term still to be determined.

The goal in Lean

$$P\ Q : \mathrm{Prop}$$
$$hp : P$$
$$hq : Q$$
$$\vdash P \wedge Q$$

The missing proof

?g : $P \wedge Q$

?g is a **metavariable**:
a hole in the proof expression.

Its local context includes P, Q, hp, and hq.

One possible assignment is ?g := `And.intro` hp hq.

The assigned term must have the target type and use only available locals.

MetaM: context and state

	Available information	What can we do?
Environment	Constant declarations	Look up `Nat.succ : Nat → Nat`. Read the body of a defined constant.
Local context	Variables and hypotheses in scope	Find the local proof `hp : P`. List the available hypotheses.
Metavariable context	Holes: types, local contexts, and assignments	Create a hole `?x : Nat`. Record an assignment `?x := 7`.

MetaM: create, assign, instantiate

Build an expression containing a hole, then fill that hole.

Step	Tool	Result
1. Create a hole	`mkFreshExprMVar`	Register `?x : Nat`.
2. Build a term	`.app`	`e = Nat.succ ?x`
3. Assign the hole	`MVarId.assign`	Record `?x := 7` in the context.
4. Substitute its value	`instantiateMVars e`	Return `Nat.succ 7`.

Instantiation reads the assignment table to replace assigned holes.

The original `e` remains unchanged.

Normal forms and definitional equality

Which definitions may unfold? *Transparency*

`.none`	Keep constant definitions folded.
`.reducible`	Unfold `abbrev` and `@[reducible]` definitions.
`.default`	Also unfold ordinary `def`.
`.all`	Also unfold `@[irreducible]` definitions.

The same expression can have different normal forms at different transparencies.

Where do we reduce?

`whnf`: reduce until the outer form is exposed.
Full normalisation: reduce inside arguments and function bodies too.

Definitional equality (`isDefEq`)

Compare up to computation; may assign metavariables to make terms match.

Elaborating a term in context

Term elaboration turns parsed syntax into a typed expression.



Local names and implicit arguments are resolved using the context and expected type.

The resulting proof term must pass the kernel's type checker.

Sums of squares (SOS)

To prove a polynomial nonnegative, write it as a sum of squares with nonnegative weights:

$$p(\mathbf{x}) = \sum_i c_i \, q_i(\mathbf{x})^2, \qquad c_i \geq 0.$$

For example,

$$x^2 + y^2 + z^2 - xy - yz - zx = \tfrac{1}{2}(x-y)^2 + \tfrac{1}{2}(y-z)^2 + \tfrac{1}{2}(z-x)^2.$$

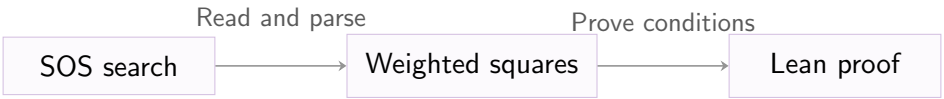
The weights and polynomials form a **certificate** of nonnegativity.

Hard to find, easy to verify

Other examples: integer factorization, satisfying assignments for SAT.

External search and checked evidence

Python calls SumOfSquares.py to find weighted squares.



Successful parsing establishes the response's format.

Lean checks nonnegative weights and the polynomial identity.

The IO call runs while elaborating the proof.